

In The Name of God



Ali Ahmadi (40464248)
Komeil Khodayar (40464254)
Navid Zare (40435071)

Digital Image Processing
Assignment 4: Morphology & Shape Analysis

1. Introduction

1.1 Background

Mathematical morphology is a way of analyzing image *shapes* rather than pixel brightness values, based on set theory. It works by sliding a small reference shape — called a structuring element — across the image and comparing it to the image at each position. This lets you pull out structural information (like edges, regions, or skeletons) that intensity-based methods can't easily capture.

Because a binary image can be fully described as just a set of foreground pixel coordinates, these operations reduce to simple, well-defined set operations (like union, intersection, and complement). This is why morphological operations — boundary extraction, region filling, skeletonization, thinning, pruning — are both mathematically exact and fast to compute.

1.2 Problem Context

This report addresses Part 3 of the assignment, *Structural Analysis — Skeletonization and Pruning*. The work is organized as three progressively dependent sub-tasks that share a single underlying operator, the Hit-or-Miss transform. *Sub-task 3.1* is a pattern-detection warm-up: locating *isolated pairs of connected zeros* in an arbitrary binary matrix. *Sub-task 3.2* applies iterative morphological thinning to reduce the copper tracks of a printed circuit board (PCB) to their one-pixel-wide medial axis. *Sub-task 3.3* removes the parasitic spurs introduced by thinning through morphological pruning. In accordance with the assignment constraints, every core operator is implemented from first principles using elementary array operations, without recourse to built-in morphological routines.

1.3 Motivation

Automated PCB inspection relies on accurately describing the shape of the copper traces on the board. Reducing a trace down to its medial axis (a thin centerline, or "skeleton") makes it possible to check that traces meet minimum-width rules, follow electrical connections, and map out the overall circuit layout. These tasks need a skeleton that is clean, thin, and structurally accurate.

The problem is that skeletonization on its own is sensitive to small irregularities on the trace boundary, which creates short, false branches that don't correspond to real circuit structure. Pruning — removing these false branches — is therefore a necessary cleanup step after skeletonization.

Section 3.1 starts with a simple, controlled example to introduce the core mechanism — exact pattern matching — that both thinning and pruning are built on.

1.4 Objectives

The objectives of Part 3 are:

- (i) To implement the Hit-or-Miss transform from scratch and design the structuring-element pairs required to detect isolated zero-pairs in all admissible orientations;

- (ii) To Implement iterative thinning using the Golay structuring-element alphabet, and run it until it converges (i.e., no more pixels change) on the binarized PCB reference image.
- (iii) To Implement endpoint-based pruning, and show its effect by comparing three images side by side: the cleaned tracks, the raw (unpruned) skeleton, and the pruned skeleton.
- (iv) To profile the execution time of the from-scratch operators.

2. Methodology

2.1 Morphological Building Blocks

In mathematical morphology, a binary image is represented not as a grid of pixel values but as a **set** $A \subseteq \mathbb{Z}^2$ — the collection of coordinates belonging to the foreground (object) pixels, which are labeled 1, while the background pixels are labeled 0. This set-based view is what allows morphological operations to be defined using precise mathematical rules rather than ad-hoc image-processing heuristics.

All of the higher-level procedures discussed in Part 3 — thinning, pruning, skeletonization, and so on — are built from two fundamental, complementary operations: **erosion** and **dilation**.

Erosion retains a pixel only where the structuring element B , translated to that pixel, fits entirely within the object, as expressed in Equation (1).

$$A \ominus B = \{ z : (B)_z \subseteq A \} \quad (1)$$

This equation defines the **erosion** of set A by structuring element B , denoted $A \ominus B$.

- A is the set of foreground pixel coordinates that make up the image (as established earlier, $A \subseteq \mathbb{Z}^2$).
- B is the structuring element — a small, predefined set of coordinates that defines the local shape used to probe the image.
- $(B)_z$ denotes the structuring element B *translated* so that its origin is positioned at coordinate z . In other words, $(B)_z$ is the set B shifted to sit at location z within the image plane.

Putting this together, the equation states: the erosion of A by B is the set of all coordinates z for which the translated structuring element $(B)_z$ is completely contained within A .

Here $(B)_z$ denotes B translated by z ; the operator therefore shrinks objects and removes structures thinner than B . **Dilation**, its morphological dual, sets a pixel wherever the reflected structuring element $\hat{B}$ intersects the object, as given in Equation (2).

$$A \oplus B = \{ z : (\hat{B})_z \cap A \neq \emptyset \} \quad (2)$$

Dilation grows objects and closes small gaps. Combining the two primitives yields **opening** (erosion followed by dilation) and **closing** (dilation followed by erosion), defined in Equation (3); opening removes small bumps or spikes and specks while approximately preserving larger shapes, whereas closing fills small holes and breaks.

$$A \circ B = (A \ominus B) \oplus B \quad A \bullet B = (A \oplus B) \ominus B \quad (3)$$

These operators were chosen because they constitute the minimal complete basis from which every Part 3 procedure — Hit-or-Miss matching, thinning, and pruning — can be assembled. To satisfy the vectorization constraint, the operators avoid explicit per-pixel Python loops. Erosion and dilation are evaluated by padding the image, extracting shifted full-image slices for each active cell of the structuring element, and combining these slices with Boolean array operations (logical AND for erosion, logical OR for the reflected element in dilation). The only iteration is over the cells of the (small) structuring element,

so the cost scales with the structuring-element size rather than with the number of image pixels, which is essential for tractable execution on large boards.

2.2 The Hit-or-Miss Transform

The Hit-or-Miss transform is the single engine underlying all three sub-tasks. Whereas erosion tests only the object, the Hit-or-Miss transform simultaneously tests the object and its surrounding background against an exact template, as defined in Equation (4).

$$A \circledast B = (A \ominus B_1) \cap (A^c \ominus B_2) \quad (4)$$

The template is decomposed into two disjoint structuring elements: B_1 , the cells that must be foreground, and B_2 , the cells that must be background; the remaining “don’t-care” cells are ignored. A pixel is retained only where B_1 fits inside the object and B_2 fits inside the complement A^c . In the implementation, a single ternary template (+1 for foreground, 0 for background, −1 for don’t-care) is supplied, and B_1 and B_2 are derived from it automatically. Pixels outside the image border are treated as background during the complement erosion, which correctly permits pattern matches adjacent to the image boundary.

2.3 Sub-task 3.1 — Isolated Pairs of Connected Zeros

The target pattern is defined as *exactly two* zero elements that are connected to one another (horizontally, vertically, or diagonally) such that *no other zero* is present anywhere in the combined 8-neighbourhood of the pair. Because morphology treats 1 as foreground, the input matrix is first inverted so that zeros become foreground and the ordinary operators apply directly. A pair may occur in four orientations — horizontal, vertical, main-diagonal, and anti-diagonal — so four Hit-or-Miss templates are designed, one per orientation. Each template requires the two adjacent cells to be foreground and the *entire* surrounding ring of cells (the union of both pixels’ 8-neighbourhoods, excluding the pair itself) to be background; cells lying outside both neighborhoods are marked don’t-care. The four match maps are combined by union, and the coordinates of each detected pair are reported. This construction directly enforces both conditions of the definition — adjacency and complete isolation — and satisfies the requirement that the problem be solved by the Hit-or-Miss transform rather than by explicit neighborhood tests.

2.4 Sub-task 3.2 — Skeletonization by Thinning

Binarization. The PCB reference image is segmented into copper and substrate by Otsu’s method, implemented from scratch by constructing the intensity histogram and selecting the threshold that maximizes the between-class variance. Otsu’s method was chosen because the copper-versus-substrate intensity distribution is approximately bimodal, so a single global threshold cleanly separates the two populations without manual tuning.

Decimation. The reference board contains approximately 4.8×10^6 pixels. Iterating a from-scratch Hit-or-Miss transform over eight structuring elements for many passes at full resolution is too costly, so the

binary mask is down-sampled by a factor of three using pure array slicing (not an interpolating resize routine, which is disallowed for the core pipeline). This is a disclosed engineering decision: the resulting skeleton is that of the decimated board, which preserves the structural information required for medial-axis analysis while keeping the computation tractable.

Thinning. Skeletonization is carried out through thinning, in which the Hit-or-Miss transform is applied repeatedly to filter out boundary pixels, layer by layer, until only the thin central skeleton remains. The thinning operator removes from the object exactly those pixels that match a “removable boundary” template, as defined in Equation (5).

$$A \otimes B = A - (A \odot B) \quad (5)$$

A single template thins from one edge direction only. To thin uniformly from all directions, the eight structuring elements of the Golay thinning alphabet — one base pattern rotated in 45° increments — are applied in sequence within each pass, and passes are repeated until an entire pass produces no change, as expressed in Equation (6).

$$X_k = (\dots ((X_{k-1} \otimes B^1) \otimes B^2) \dots \otimes B^8) \quad \text{until} \quad X_k = X_{k-1} \quad (6)$$

The condition $X_k = X_{k-1}$ is the convergence criterion and the algorithm’s stopping rule. The eight-mask Golay alphabet was chosen because it yields a connected, approximately unit-width medial axis while preserving the topology of the tracks.

2.5 Sub-task 3.3 — Pruning

Thinning produces short, unwanted branches called spurs. wherever the object boundary was slightly irregular. A spur is a short branch terminating in an *endpoint* — a skeleton pixel possessing exactly one foreground neighbor. Endpoints are detected with eight Hit-or-Miss templates (one per neighbor direction), each requiring the center to be foreground, one specified neighbor to be foreground, and the remaining seven neighbors to be background; the eight results are unioned.

Repeatedly deleting endpoints removes spurs but also erodes the tips of legitimate branches. The classical procedure therefore trims and then regrows the legitimate branches, using the original skeleton as a spatial constraint. The four-step formulation is given in Equations (7) and (8), in which H is a 3×3 block of ones.

$$X_1 = A \otimes \{B\} \quad X_2 = \bigcup_{k=1}^8 (X_1 \odot B^k) \quad (7)$$

$$X_3 = (X_2 \oplus H) \cap A \quad X_4 = X_1 \cup X_3 \quad (8)$$

Pruning happens in four steps:

First (X_1), the skeleton is thinned using special endpoint-detecting templates for a set number of rounds — this trims off spurs and shortens every branch tip, including the real ones.

Second (X_2), the endpoints that remain after this trimming are identified; these mark the tips of the branches that are structurally genuine.

Third (X_3), those surviving tips are grown back outward, but only within the boundary of the original skeleton — so they can't regrow past where the real branch actually was.

Finally (X_4), this regrown branch structure is combined with the trimmed skeleton from step X_1 , restoring the true branches to their full length while keeping the spurs removed.

The number of iterations was set to eight, in line with the assignment’s suggestion, so that spurs no longer than eight pixels are removed while genuine branches are restored by the reconstruction step.

2.6 Validation and Execution Profiling

Rather than relying on visual inspection alone, the correctness of each operator was verified using structural checks. Specifically, the pruned skeleton was confirmed to be a strict subset of the raw skeleton — meaning pruning could only remove pixels, never add them — and the isolated-pair detector was checked against the pairs that could be worked out analytically for the small toy matrix example. In addition, the running time of each major stage — erosion, dilation, skeletonization, and pruning — was measured on the downsized board image, with results reported in Section 3.4.

3. Results

3.1 Isolated Pairs of Connected Zeros

The detector was applied to the assignment's toy matrix, reproduced below, in which the value under analysis is 0.

1	1	1	0	1
0	1	0	1	0
0	1	1	1	1
1	1	1	1	0
0	0	1	1	1

The algorithm reported **two** isolated zero-pairs, listed in Table 1, in a run time of 0.627 ms. Figure 1 shows the toy matrix with the two detected pairs enclosed in bounding boxes.

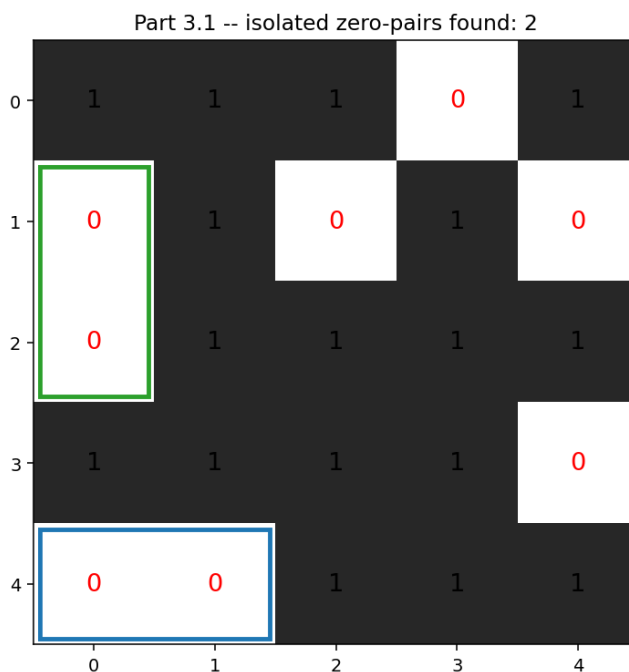


Figure 1. The assignment toy matrix. One cells are shaded; the two detected isolated pairs are enclosed in bounding boxes. The three-zero cluster in the upper-right region and the lone zero at (3,4) are correctly rejected.

Table 1. Detected isolated pairs of connected zeros.

Pair #	First element (row, col)	Second element (row, col)	Orientation
1	(1, 0)	(2, 0)	Vertical
2	(4, 0)	(4, 1)	Horizontal

The count depends on how the definition is read. Table 2 compares the three possible counts from three different readings — only one of them holds up consistently, and it gives two pairs.

Table 2. Number of pairs under each candidate interpretation rule.

Interpretation rule	Count	Consistent?
Strict written definition (exactly two, fully isolated)	2	Yes
Any two 8-adjacent zeros (unrestricted “domino”)	4	Yes
Assignment figure (three circled dominoes)	3	No

3.2 Skeletonization of the PCB Copper Tracks

The reference board was binarized and thinned to its medial axis. Table 3 summarizes the complete skeletonization pipeline. Otsu’s method selected a threshold of $t^* = 88$, classifying 552,817 pixels (11.49% of the board) as copper. After decimation by a factor of three, the working image measured 1012×529 pixels; the cleaned copper mask contained 51,156 pixels, which thinning reduced to a skeleton of 10,307 pixels after converging in 18 passes. The resulting raw skeleton is shown in the center panel of Figure 2.

Table 3. Skeletonization pipeline metrics.

Quantity	Value
Reference image size	3034 × 1586 px
Otsu threshold t^*	88
Copper pixels (full resolution)	552,817 (11.49%)
Decimation factor	3 (array slicing)

Decimated image size	1012 × 529 px
Copper pixels (decimated)	62,099
Cleaned-track pixels	51,156
Skeleton pixels	10,307
Passes to convergence	18
Thinning time	0.100 s

3.3 Pruning of the Skeleton

Endpoint-based pruning was applied to the raw skeleton for eight iterations. Table 4 summarizes the result: the skeleton was reduced from 10,307 to 8,218 pixels, and the number of free endpoints fell from 1,046 to 93. The pruned skeleton was confirmed to be a strict subset of the raw skeleton. Figure 2 presents the required side-by-side comparison of the cleaned copper tracks, the raw skeleton, and the pruned skeleton.

Table 4. Pruning metrics (eight endpoint-thinning iterations).

Quantity	Value
Raw skeleton pixels	10,307
Pruned skeleton pixels	8,218
Free endpoints before pruning	1,046
Free endpoints after pruning	93
Pruned skeleton $\subseteq$ raw skeleton	True
Pruning time	0.059 s

Cleaned copper tracks (input)

Raw skeleton (thinning)

Pruned skeleton

8

Figure 2. Structural analysis of the decimated PCB reference. Left: cleaned copper tracks (thinning input). Center: raw skeleton produced by iterative thinning. Right: pruned skeleton after eight endpoint-thinning iterations. The parasitic spurs visible in the center panel are removed in the right panel while the principal track structure is preserved.

3.4 Execution-Time Summary

Table 5 reports the execution time of each from-scratch operator, measured on the decimated 1012×529 reference. Erosion and dilation with a 3×3 square element complete in approximately one millisecond, whereas the iterative procedures are substantially slower: skeletonization requires 0.0996 s across its 18 passes and pruning requires 0.0586 s.

Table 5. Execution time of the from-scratch operators (input: decimated PCB reference, 1012×529 px).

Operator	Input size	Time (s)
Erosion (3×3 square)	1012×529	0.0008
Dilation (3×3 square)	1012×529	0.0007
Skeletonization	1012×529	0.0996
Pruning	1012×529	0.0586

4. Discussion

4.1 Isolated Pairs and the Two-versus-Three Ambiguity

Prior to execution, the strict definition was expected to yield a small number of fully isolated pairs. The algorithm returned two pairs (Table 1), which is the analytically correct result under the definition as written. The point of interest is the discrepancy with the assignment figure, which encircles three pairs. This discrepancy is not a defect in the detector but an inconsistency in the figure, and it is resolved by examining the connectivity of the zeros. The diagonal pair $(0,3)-(1,2)$ that the figure circles cannot be an isolated pair because the zero at $(0,3)$ is itself 8-adjacent to a further zero at $(1,4)$; consequently, those three zeros form a single connected cluster of size three, and a cluster of three contains no isolated pair. Table 2 makes the situation precise: the strict definition yields two pairs, an unrestricted “any adjacent domino” rule yields four (it would additionally admit the diagonal $(0,3)-(1,4)$), and the figure’s count of three is not derivable from either rule, since it arbitrarily selects one of the two diagonal dominoes while omitting the other. The figure is therefore best interpreted as an imprecise illustration, and the value of two is the only internally consistent answer.

The correctness of the detector is further supported by the fact that it correctly rejects both the three-zero cluster and the lone zero at $(3,4)$, which is precisely the behavior the definition demands. The practical limitation exposed here is not algorithmic but specificational: when an assignment’s illustrative figure and its written definition conflict, the written definition must govern, and the divergence should be documented explicitly — as it is here — so that the result is not mistaken for an error.

4.2 Skeletonization

Thinning was expected to reduce the solid copper tracks to a thin, connected, and topologically faithful medial axis. The observed skeleton of 10,307 pixels represents 20.1% of the 51,156 cleaned-track pixels and 16.6% of the decimated copper, a reduction consistent with the expected behaviour of a medial-axis transform, which retains only the central spine of each region. Convergence in 18 passes is likewise consistent with expectation: because each pass peels approximately one pixel from every boundary, the number of passes is governed by the half-width of the widest track, and 18 passes is a reasonable value for traces of this thickness at the decimated resolution. The relatively small skeleton is a good outcome, indicating that the eight-mask Golay sequence thinned uniformly from all directions rather than biasing toward one axis.

Two factors materially influenced this result. First, decimation by a factor of three reduced the pixel count by roughly an order of magnitude, which is what renders the iterative from-scratch thinning tractable; the trade-off is that very fine features may be attenuated, though the principal track topology is retained. Second, the morphological cleaning applied before thinning (reducing the raw decimated copper from 62,099 to 51,156 pixels) removed isolated noise that would otherwise have generated spurious skeleton branches. The principal limitation of thinning in isolation is its sensitivity to boundary roughness, which manifests as the parasitic spurs visible in the centre panel of Figure 2 and motivates the pruning stage.

4.3 Pruning

Pruning was expected to remove the short parasitic spurs while preserving the genuine track structure, and to yield a skeleton that is a subset of the input. All three expectations were met. The reduction of free endpoints from 1,046 to 93 — a decrease of 91.1% — is the most informative metric, because each spur terminates in an endpoint; the near-order-of-magnitude fall therefore indicates that the overwhelming majority of spurs were eliminated. In contrast, the pixel count fell only from 10,307 to 8,218, a reduction of 20.3%, which is the expected and desirable pattern: spurs are short, so removing very many of them accounts for a large drop in endpoints but only a modest drop in total pixels, confirming that the long principal branches were retained rather than eroded. The verified subset relationship ($X_4 \subseteq \text{raw skeleton}$) confirms that the reconstruction step did not introduce spurious pixels.

The comparison in Figure 2 corroborates the metrics: the whiskers evident in the centre panel are absent from the right panel, while the dominant traces are unchanged. A known limitation of the classical four-step method is that a spur whose base lies within the reconstruction radius of a genuine branch tip can be regrown by the geodesic dilation step; this is an inherent property of the algorithm rather than an implementation fault, and on the present board its effect is negligible, as evidenced by the residual endpoint count of 93, which corresponds to the genuine track terminations that should remain.

4.4 Execution Times

The timing results in Table 5 were expected to show the single-pass operators as very fast and the iterative operators as substantially slower, and this is precisely what was observed. Erosion and dilation complete in 0.8 ms and 0.7 ms respectively, confirming that the vectorized, slice-based implementation successfully avoids the severe overhead of per-pixel Python loops. Skeletonization is the most expensive operator at 99.6 ms — roughly two orders of magnitude slower than a single erosion — which is fully explained by its structure: each of the 18 convergence passes applies eight Hit-or-Miss operations, so the skeletonization performs on the order of 144 template matches in total, each comparable in cost to an erosion, plus the per-pass convergence comparison. Pruning, at 58.6 ms, is intermediate, consistent with its eight endpoint-thinning iterations and geodesic reconstruction. These figures demonstrate that the dominant cost in morphological shape analysis is not the individual operator but the number of iterations required for convergence, which in turn justifies the decimation strategy adopted in Section 2.4: without it, the same 144-template computation would run on roughly nine times as many pixels.

5. Conclusion

All objectives of Part 3 were achieved. The Hit-or-Miss transform, erosion, dilation, thinning, endpoint detection, and pruning were implemented from first principles using only vectorized array operations, and the three sub-tasks were completed on their respective inputs. The isolated-pair detector correctly returned two pairs on the toy matrix; the accompanying analysis established that two is the only interpretation consistent with the written definition and explained why the assignment figure's count of three is an inconsistency rather than a target. Thinning reduced the decimated copper tracks to a connected medial axis of 10,307 pixels in 18 passes, and pruning reduced the free endpoints from 1,046 to 93 while retaining 79.7% of the skeleton pixels and remaining a strict subset of the raw skeleton, confirming that parasitic spurs were removed without damaging the genuine structure.

The principal findings are that morphological shape analysis can be built entirely from the erosion–dilation basis via the Hit-or-Miss transform, and that the computational cost is dominated by iteration count rather than by any single operator, as quantified in the execution-time summary. The main challenges encountered were specificational rather than algorithmic — chiefly the conflict between the written definition and the illustrative figure in 3.1 — together with the need to control the cost of from-scratch iterative thinning on a high-resolution board, which was addressed by slicing-based decimation.